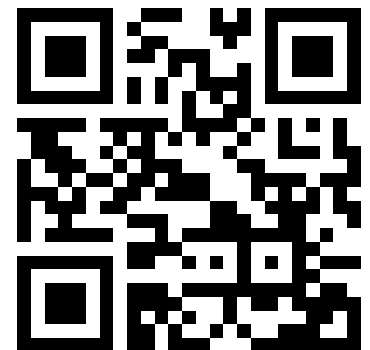


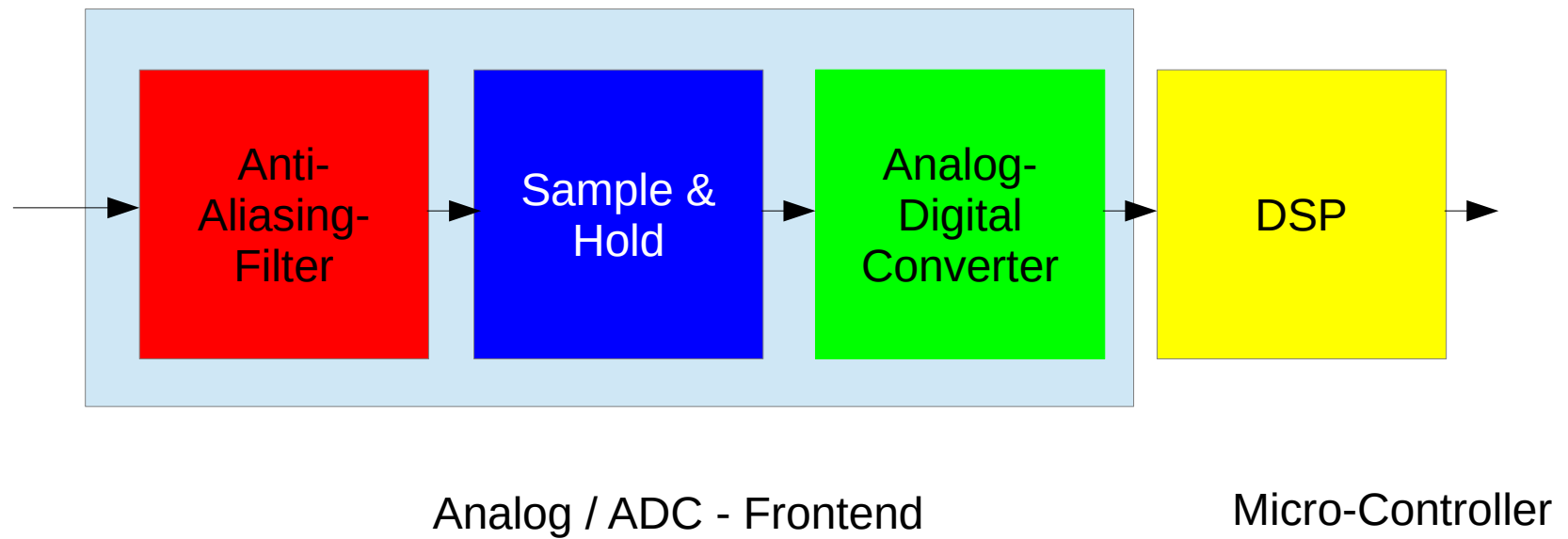
Lecture Overview

- Introduction, Microcontroller basics
- 32-bit Microcontroller technology
- ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
- Cortex M Microcontroller Instruction Set
- Exception Handling, Interrupts
- Real-time Operating Systems (RTOS)
- Memory Management, protected memory, MPU
- Implementing Calculations, Data Formats, FPU
- I/O, Device drivers
- Application Design
- Case Studies, Advanced Design Patterns

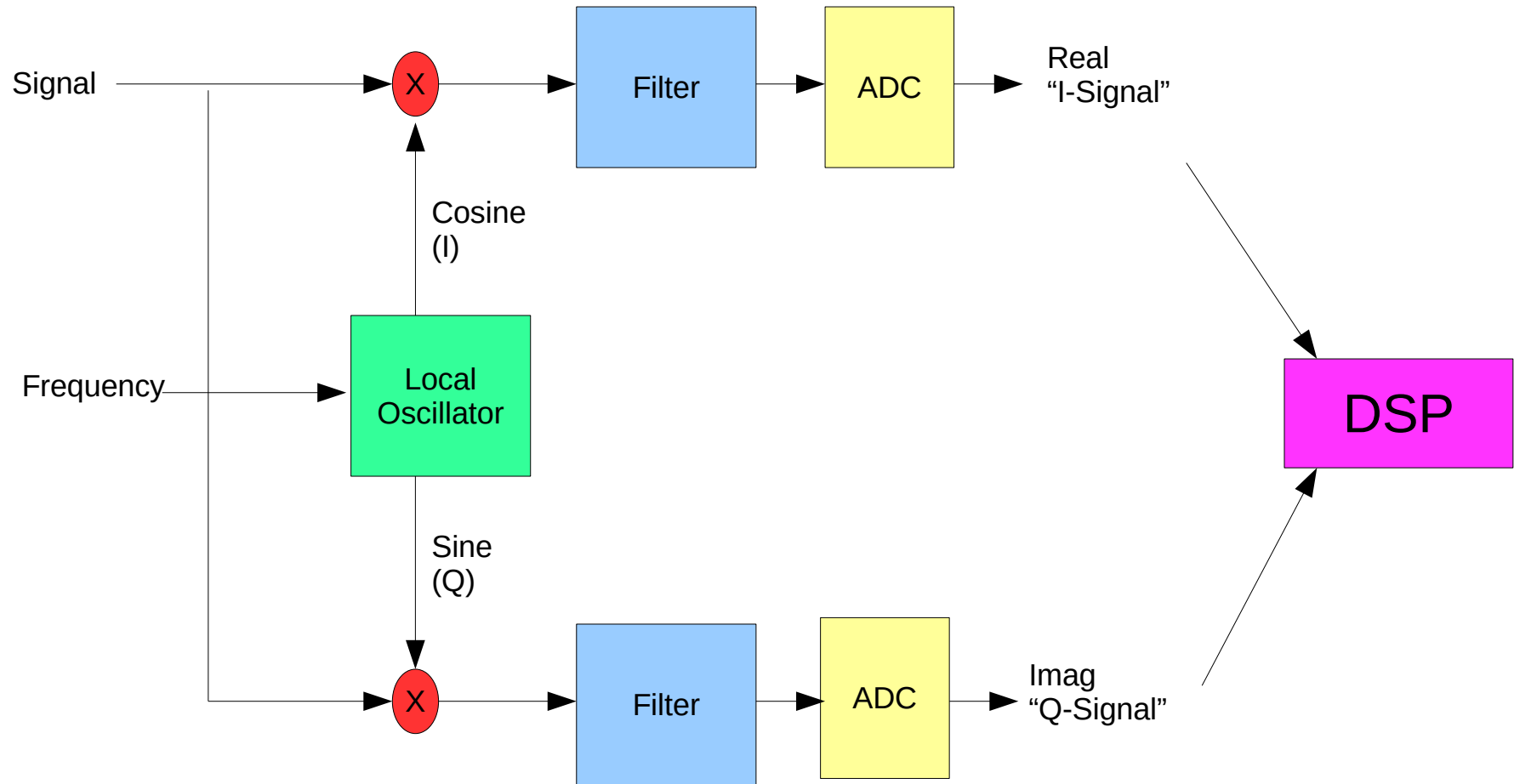
<https://skript.eit.h-da.de/ams/>



DSP Systems



Software-Defined Radio



Digital Signal Processing

- Signed and unsigned **integer** numbers
CM4 supports 8 / 16 / 32 and 64 bit result formats
- **Fractional** numbers
CM4 supports 8 / 16 / 32 / 64 bit number formats
- **Floating point** numbers
CM4 offers hardware support
for 32 bit single precision floating point numbers
Some types of CM7 have a 64-bit FPU

Cortex-M4 FPU

VABS	VADD	VCMP	VCMPPE	VCVT	VCVTR	VDIV	VLDM
VLDR	VMLA	VMLS	VMOV	VMRS	VMSR	VMUL	VNEG
VNMLA	VMMLS	VNMUL	VPOP	VPUSH	VSQRT	VSTM	VSTR
VSUB	VFMA	VFMS	VFNMA	VFNMS			
PKH	QADD	QADD16	QADD8	QASX	QDADD	QDSUB	QSAX
QSUB	QSUB16	QSUB8	SADD16	SADD8	SASX	SEL	SHADD16
SHADD8	SHASX	SHSAX	SHSUB16	SHSUB8	SMLABB	SMLABT	SMLATB
SMLATT	SMLAD	SMLALBB	SMLALBT	SMLALTB	SMLALTT	SMLALD	SMLAWB
SMLAWT	SMLS	SMLS	SMMLA	SMMLS	SMMUL	SMUAD	SMULBB
ADC	ADD	ADR	AND	ASR	B	SMULBT	SMULTT
CLZ	BFC	BF	BIC	CDP	CLREX	SMULTB	SMULWT
CBNZ	CBZ	CMN	CMP	DBG	EOR	SMULWB	SMUSD
LDMIA	LDMDB	LDR	LDRB	LDRBT	LDRD	SSAT16	SSAX
LDREX	LDREXB	LDREXH	LDRH	LDRHT	LDRSB	SSUB16	SSUB8
LDRSBT	LDRSHT	LDRSH	LDRT	MCR	LSL	SXTAB	SXTAB16
LSR	MCRR	MLS	MLA	MOV	MOVT	SXTAH	SXTB16
MRC	MRRC	MUL	MVN	NOP	ORN	UADD16	UADD8
ORR	PLD	PLDW	PLI	POP	PUSH	UASX	UHADD16
RBIT	REV	REV16	REVSH	ROR	RRX	UHADD8	UHASX
			RSB	SBC	SBFX	UHSAX	UHSUB16
			SDIV	SEV	SMLAL	UHSUB8	UMAAL
			SMULL	SSAT	STC	UQADD16	UQADD8
			STMIA	STMDB	STR	UQASX	UQSAX
			STRB	STRBT	STRD	UQSUB16	UQSUB8
			STREX	STREXB	STREXH	USAD8	USADA8
			STRH	STRHT	STRT	USAT16	USAX
			SUB	SXTB	SXTH	USUB16	USUB8
			TBB	TBH	TEQ	UXTAB	UXTAB16
			TST	UBFX	UDIV	UXTAH	UXTB16
			UMLAL	UMULL	USAT		
			UXTB	UXTH	WFE		
			WFI	YIELD	IT		
BKPT	BLX	ADC	ADD	ADR			
BX	CPS	AND	ASR	B			
DMB		BL	BIC				
DSB	CMN	CMP	EOR				
ISB	LDR	LDRB	LDM				
MRS	LDRH	LDRSB	LDRSH				
MSR	LSL	LSR	MOV				
NOP	REV	MUL	MVN	ORR			
REV16	REVSH	POP	PUSH	ROR			
SEV	SXTB	RSB	SBC	STM			
SXTH	UXTB	STR	STRB	STRH			
UXTH	WFE	SUB	SVC	TST			
WFI	YIELD						

Cortex-M0/M0+/M1

Cortex-M3

Cortex-M4

Number Formats

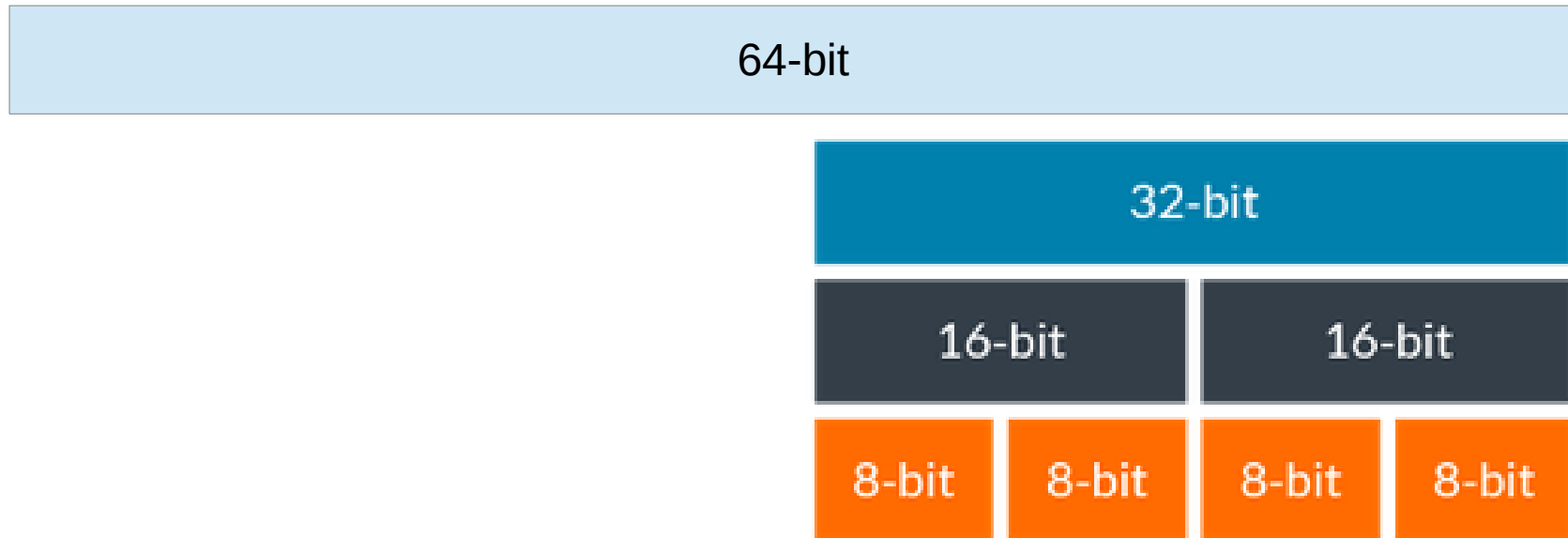
Signed Integers	
int8_t	8-bit
int16_t	16-bit
int32_t	32-bit
int64_t	64-bit

Unsigned Integers	
uint8_t	8-bit
uint16_t	16-bit
uint32_t	32-bit
uint64_t	64-bit

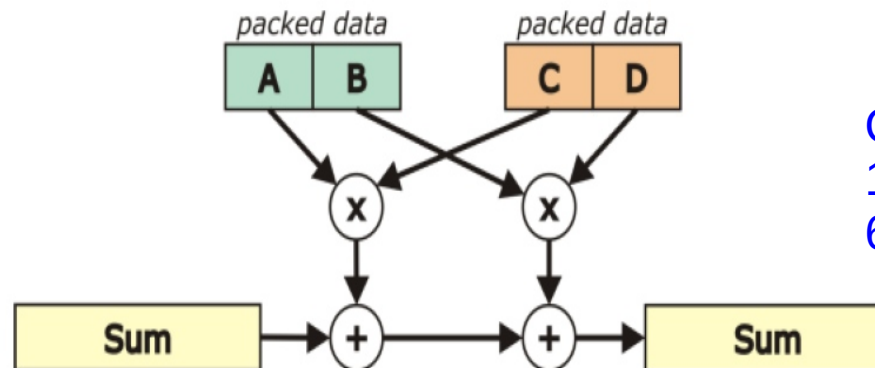
Fractional	
q7_t	8-bit
q15_t	16-bit
q31_t	32-bit
q63_t	64-bit

Floating point	
float32_t	SP 32-bit
float64_t	DP 64-bit

CM4/CM7 Built-in Number Formats



Example:

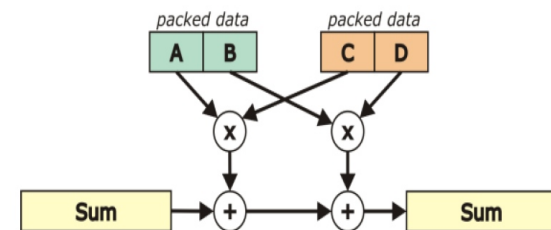


Only one command “SMLALD”
16 bit input data A B C D
64 bit accumulator for Sum

Integer computation

Counting / arithmetics with integer numbers as usual

- Does not really match the needs of DSP algorithms
- Overflow in add / sub operations may occur
- Overflow in multiply operations is always problematic
- CM4 hardware supports saturating arithmetic
- CM4 hardware directly supports 8 / 16 / 32 / 64 bit data width
- Several parallel (SIMD) computing instructions available



ADD, ADC, SUB, SBC, and RSB

Add, Add with carry, Subtract, Subtract with carry, and Reverse Subtract.

Syntax

op{*S*}{*cond*} {*Rd*,} *Rn*, *Operand2*

op{*cond*} {*Rd*,} *Rn*, #*imm12* ; ADD and SUB only

where:

<i>op</i>	Is one of: ADD Add. ADC Add with Carry. SUB Subtract. SBC Subtract with Carry. RSB Reverse Subtract.
<i>S</i>	Is an optional suffix. If <i>S</i> is specified, the condition code flags are updated on the result of the operation,
<i>cond</i>	Is an optional condition code,
<i>Rd</i>	Specifies the destination register. If <i>Rd</i> is omitted, the destination register is <i>Rn</i> .
<i>Rn</i>	Specifies the register holding the first operand.
<i>Operand2</i>	Is a flexible second operand.
<i>imm12</i>	Is any value in the range 0-4095.

Multiply and divide

Multiply	Multiply	MUL Rd, Rn, Rm	1
	Multiply accumulate	MLA Rd, Rn, Rm	1
	Multiply subtract	MLS Rd, Rn, Rm	1
	Long signed	SMULL RdLo, RdHi, Rn, Rm	1
	Long unsigned	UMULL RdLo, RdHi, Rn, Rm	1
	Long signed accumulate	SMLAL RdLo, RdHi, Rn, Rm	1
	Long unsigned accumulate	UMLAL RdLo, RdHi, Rn, Rm	1
Divide	Signed	SDIV Rd, Rn, Rm	2 to 12 ^a
	Unsigned	UDIV Rd, Rn, Rm	2 to 12 ^a

CM4 directly supports “long” 64-bit multiplication results !
Single cycle multiply, and 2 - max. 12 cycle division.

Saturating numbers

if($x > \text{MAX}$) $x = \text{MAX}$

SSAT and USAT

Signed Saturate and Unsigned Saturate to any bit position, with optional shift before saturating.

Syntax

op{*cond*} *Rd*, #*n*, *Rm* {, *shift* #*s*}

where:

<i>op</i>	Is one of: SSAT Saturates a signed value to a signed range. USAT Saturates a signed value to an unsigned range.
<i>cond</i>	Is an optional condition code, see
<i>Rd</i>	Specifies the destination register.
<i>n</i>	Specifies the bit position to saturate to: • <i>n</i> ranges from 1 to 32 for SSAT • <i>n</i> ranges from 0 to 31 for USAT.
<i>Rm</i>	Specifies the register containing the value to saturate.
<i>shift</i> # <i>s</i>	Is an optional shift applied to <i>Rm</i> before saturating. It must be one of the following: ASR # <i>s</i> where <i>s</i> is in the range 1 to 31 LSL # <i>s</i> where <i>s</i> is in the range 0 to 31.

Saturating arithmetics

Mnemonic	Brief description
SSAT	Signed Saturate
SSAT16	Signed Saturate Halfword
USAT	Unsigned Saturate
USAT16	Unsigned Saturate Halfword
QADD	Saturating Add
QSUB	Saturating Subtract
QSUB16	Saturating Subtract 16
QASX	Saturating Add and Subtract with Exchange
QSAX	Saturating Subtract and Add with Exchange
QDADD	Saturating Double and Add
QDSUB	Saturating Double and Subtract
UQADD16	Unsigned Saturating Add 16
UQADD8	Unsigned Saturating Add 8
UQASX	Unsigned Saturating Add and Subtract with Exchange
UQSAX	Unsigned Saturating Subtract and Add with Exchange
UQSUB16	Unsigned Saturating Subtract 16
UQSUB8	Unsigned Saturating Subtract 8

All combinations of
signed / unsigned
8 / 16 / 32 bit
ADD and SUB

Adding without ever leaving the range

SHADD16 and SHADD8

Signed Halving Add 16 and Signed Halving Add 8.

Syntax

op{*cond*}{*Rd*,} *Rn*, *Rm*

where:

<i>op</i>	Is any of: SHADD16 Signed Halving Add 16 SHADD8 Signed Halving Add 8
<i>cond</i>	Is an optional condition code, see
<i>Rd</i>	Specifies the destination register.
<i>Rn</i>	Specifies the first operand register.
<i>Rm</i>	Specifies the second operand register.

Operation

Use these instructions to add 16-bit and 8-bit data and then to halve the result before writing the result to the destination register:

CM4 / CM7

Single-Cycle DSP instructions

OPERATION	INSTRUCTIONS	CM3	CM4
$16 \times 16 = 32$	SMULBB, SMULBT, SMULTB, SMULTT	n/a	1
$16 \times 16 + 32 = 32$	SMLABB, SMLABT, SMLATB, SMLATT	n/a	1
$16 \times 16 + 64 = 64$	SMLALBB, SMLALBT, SMLALTB, SMLALTT	n/a	1
$16 \times 32 = 32$	SMULWB, SMULWT	n/a	1
$(16 \times 32) + 32 = 32$	SMLAWB, SMLAWT	n/a	1
$(16 \times 16) \pm (16 \times 16) = 32$	SMUAD, SMUADX, SMUSD, SMUSDX	n/a	1
$(16 \times 16) \pm (16 \times 16) + 32 = 32$	SMLAD, SMLADX, SMLSD, SMLSDX	n/a	1
$(16 \times 16) \pm (16 \times 16) + 64 = 64$	SMLALD, SMLALDX, SMLSLD, SMLSLDX	n/a	1
$32 \times 32 = 32$	MUL	1	1
$32 \pm (32 \times 32) = 32$	MLA, MLS	2	1
$32 \times 32 = 64$	SMULL, UMULL	5-7	1
$(32 \times 32) + 64 = 64$	SMLAL, UMLAL	5-7	1
$(32 \times 32) + 32 + 32 = 64$	UMAAL	n/a	1
$32 \pm (32 \times 32) = 32$ (upper)	SMMLA, SMMLAR, SMMLS, SMMLSR	n/a	1
$(32 \times 32) = 32$ (upper)	SMMUL, SMMULR	n/a	1

Fractional numbers

Idea: Map signed or unsigned integer range into $[-1 .. +1)$

8 bit: $x = N / 256$ (unsigned) or $x = N / 128$ (signed)

16 bit: $x = N / 65536$ (unsigned) or $x = N / 32768$ (signed)

Example (16 bits signed fractional format “q15”):

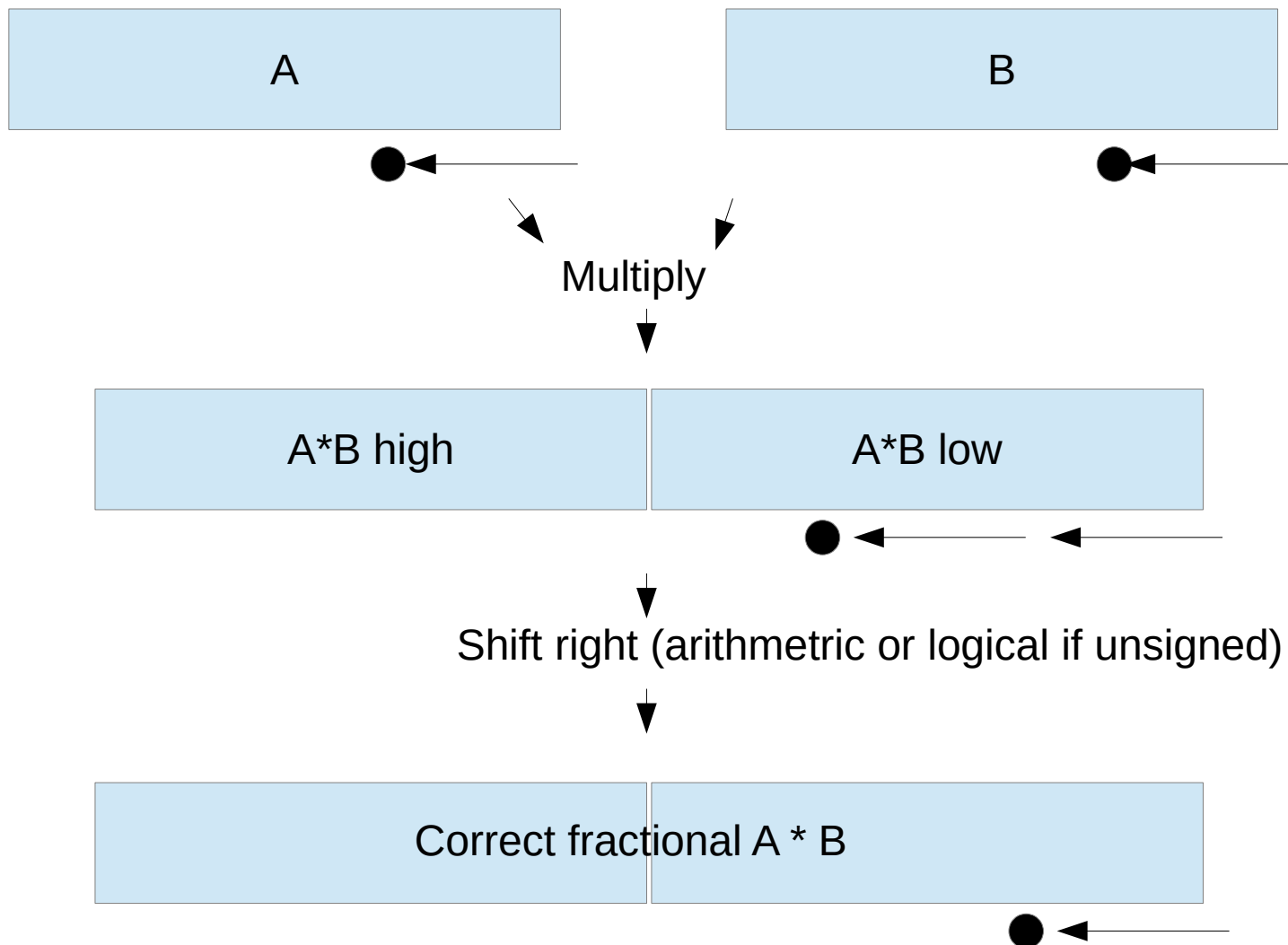
0 $\rightarrow$ 0.0

0x7fff $\rightarrow$ +1 – epsilon (exactly $32767/32768 = 0.999695\dots$)

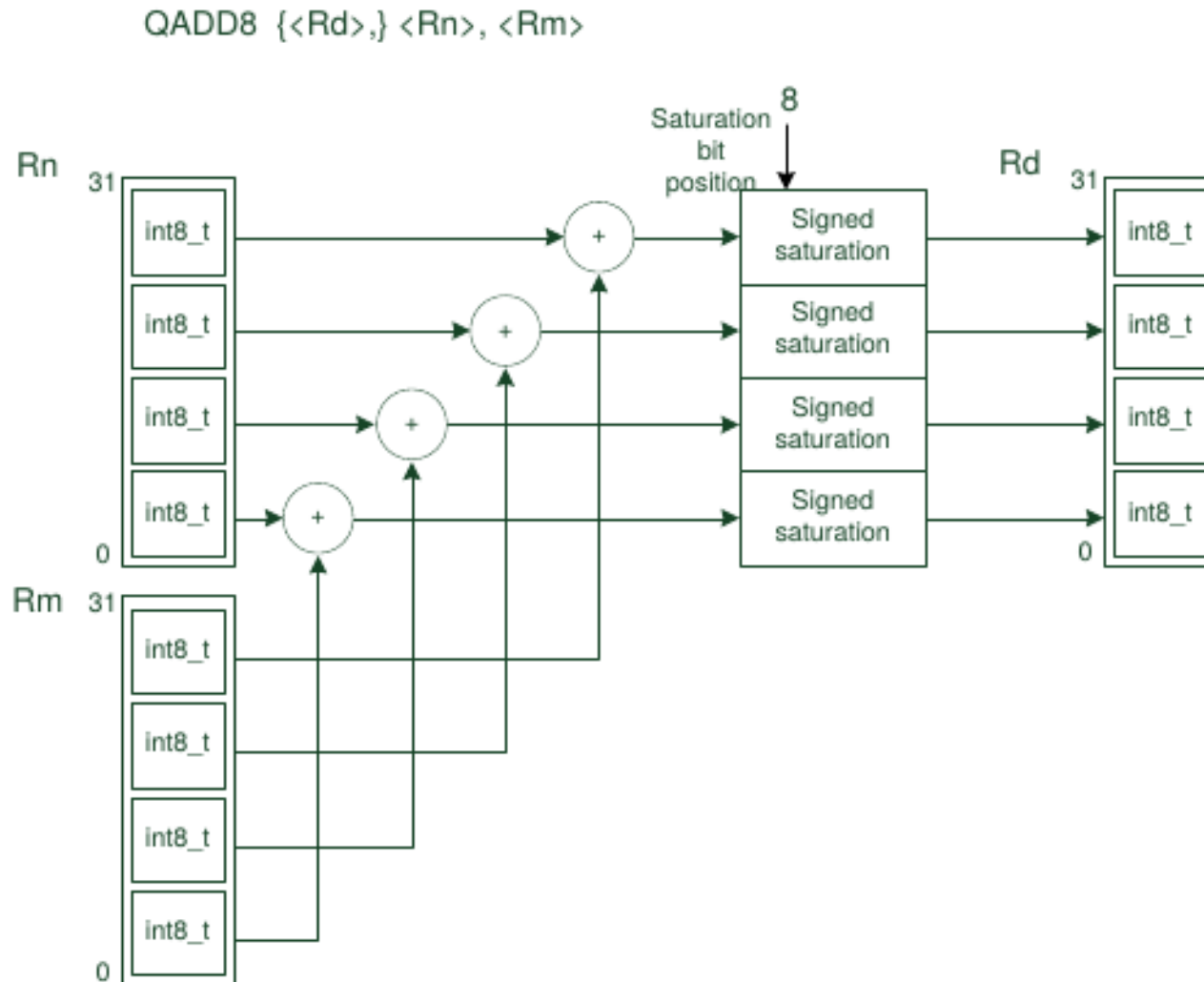
0x8000 $\rightarrow$ -1

- Multiplication is safe because the result can never leave the range.
- After the (integer) multiplication a shift needs to be done
- Additions and subtractions are dangerous as they can leave the range.
- Safe fractional computing can be done using saturating arithmetic

Fractional Multiply

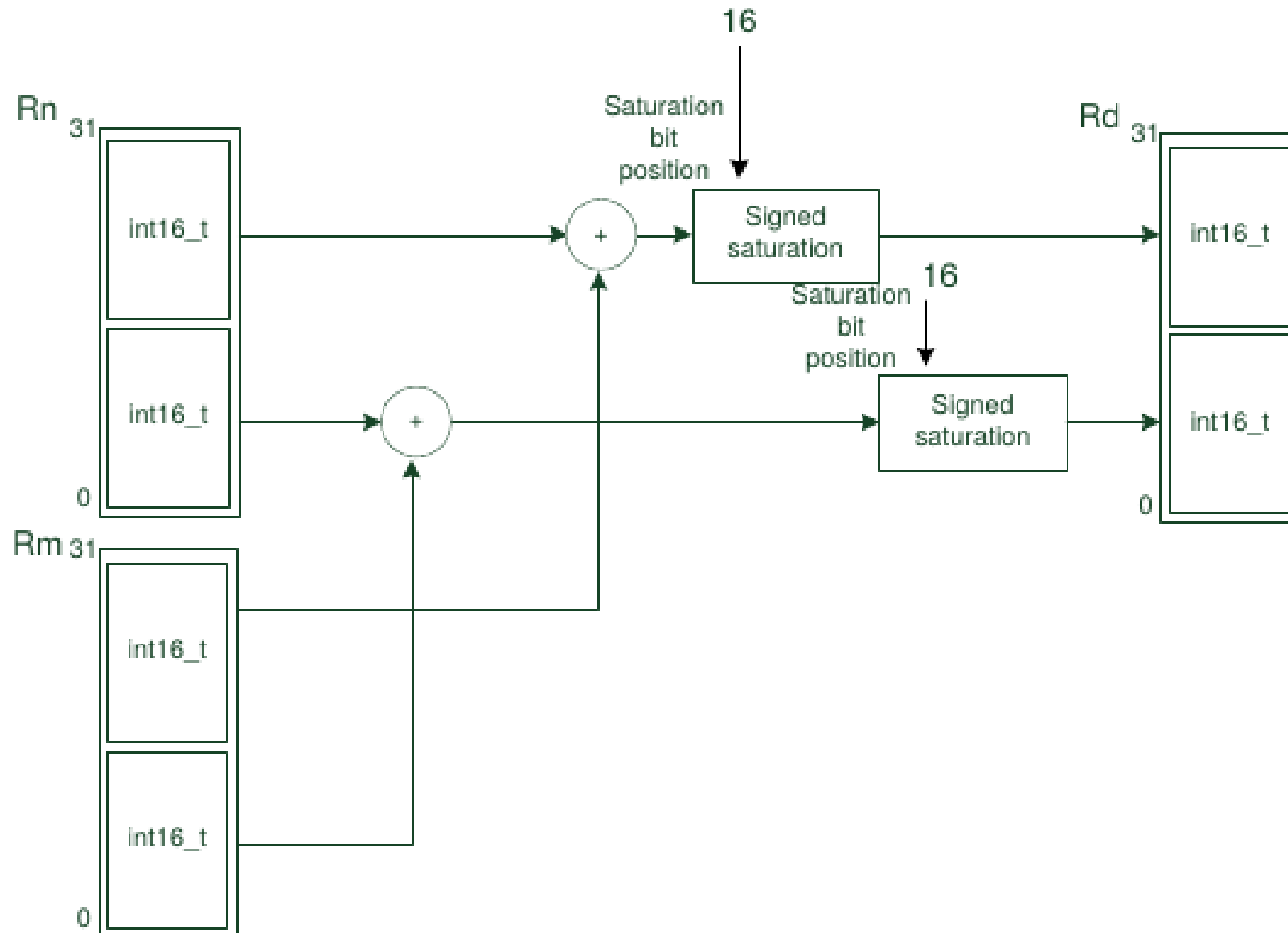


Single Instruction Multiple Data



QADD16

QADD16 {<Rd>,} <Rn>, <Rm>



Floating Point Support

Cortex M4 contains a FPU with support for

- Add
- Subtract
- Multiply
- Divide
- Multiply and accumulate
- Square root

The number format is single precision (32 bits)

Float number ranges

IEEE single precision (32 bit)

float

$\pm 10^{-37} \dots \pm 10^{+38}$

Mantissa: 23 bit = approx. 7 significant decimal digits

IEEE double precision (64 bit)

double

$\pm 10^{-307} \dots \pm 10^{+308}$

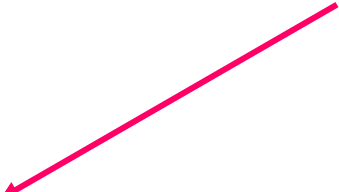
Mantissa: 52 bit = 15.65 significant decimal digits

(Distance Earth-Moon: 384401 km / $2^{52} = 0.085$ nm !)

Converting fix-point into float

Z= 1101001001 . 100010010010101001010001

Z= 1 . 101001001100010010010101001010001 * 2⁹



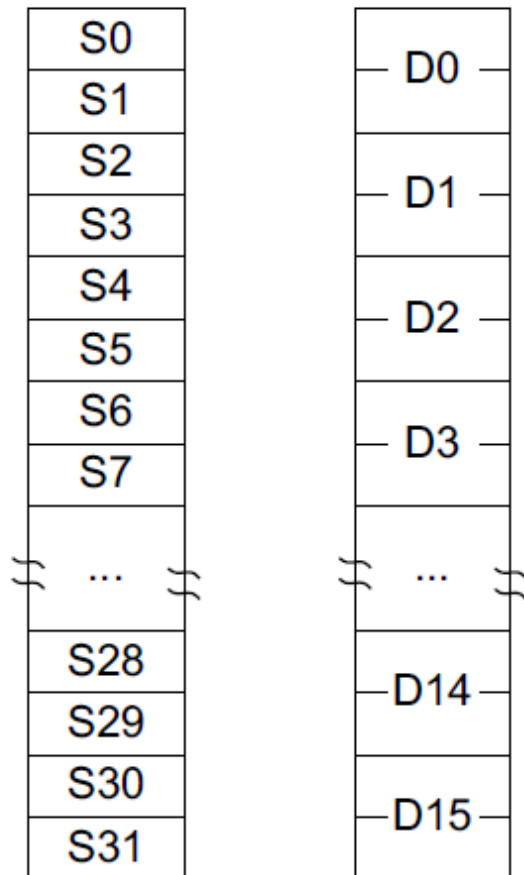
Mantissa

= 1 + 0.101001001100010010010101001010001

Exponent = 9

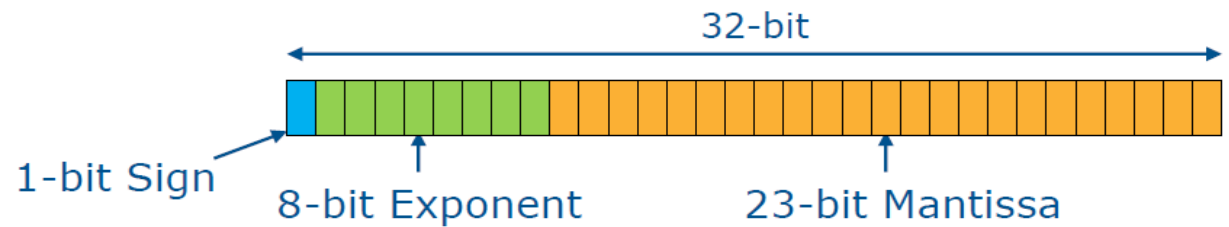
“Normalisation”

Floating Point Support

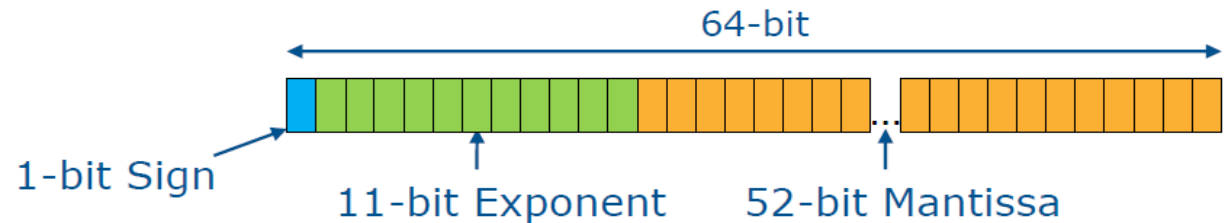


FPU register bank

Single precision : 32-bit coding



Double precision : 64-bit coding



Cortex M4F adds 32 16-bit float registers to the integer register bank.

Cortex-M4 FPU

VABS	VADD	VCMP	VCMPPE	VCVT	VCVTR	VDIV	VLDM
VLDR	VMLA	VMLS	VMOV	VMRS	VMSR	VMUL	VNEG
VNMLA	VMMLS	VNMUL	VPOP	VPUSH	VSQRT	VSTM	VSTR
VSUB	VFMA	VFMS	VFNMA	VFNMS			
PKH	QADD	QADD16	QADD8	QASX	QDADD	QDSUB	QSAX
QSUB	QSUB16	QSUB8	SADD16	SADD8	SASX	SEL	SHADD16
SHADD8	SHASX	SHSAX	SHSUB16	SHSUB8	SMLABB	SMLABT	SMLATB
SMLATT	SMLAD	SMLALBB	SMLALBT	SMLALTB	SMLALTT	SMLALD	SMLAWB
SMLAWT	SMLS	SMLS	SMMLA	SMMLS	SMMUL	SMUAD	SMULBB
ADC	ADD	ADR	AND	ASR	B	SMULBT	SMULTT
CLZ	BFC	BF	BIC	CDP	CLREX	SMULTB	SMULWT
CBNZ	CBZ	CMN	CMP	DBG	EOR	SMULWB	SMUSD
LDMIA	LDMDB	LDR	LDRB	LDRBT	LDRD	SSAT16	SSAX
LDREX	LDREXB	LDREXH	LDRH	LDRHT	LDRSB	SSUB16	SSUB8
LDRSBT	LDRSHT	LDRSH	LDRT	MCR	LSL	SXTAB	SXTAB16
LSR	MCRR	MLS	MLA	MOV	MOVT	SXTAH	SXTB16
MRC	MRRC	MUL	MVN	NOP	ORN	UADD16	UADD8
ORR	PLD	PLDW	PLI	POP	PUSH	UASX	UHADD16
RBIT	REV	REV16	REVSH	ROR	RRX	UHADD8	UHASX
			RSB	SBC	SBFX	UHSAX	UHSUB16
			SDIV	SEV	SMLAL	UHSUB8	UMAAL
BKPT	BLX	ADC	ADD	ADR		UQADD16	UQADD8
BX	CPS	AND	ASR	B		UQASX	UQSAX
DMB		BL	BIC			UQSUB16	UQSUB8
DSB	CMN	CMP	EOR			USAD8	USADA8
ISB	LDR	LDRB	LDM			USAT16	USAX
MRS	LDRH	LDRSB	LDRSH			USUB16	USUB8
MSR	LSL	LSR	MOV			UXTAB	UXTAB16
NOP	REV	MUL	MVN	ORR		UXTAH	UXTB16
REV16	REVSH	POP	PUSH	ROR			
SEV	SXTB	RSB	SBC	STM			
SXTH	UXTB	STR	STRB	STRH			
UXTH	WFE	SUB	SVC	TST			
WFI	YIELD						

Cortex-M0/M0+/M1

Cortex-M3

Cortex-M4

Floating point instruction set

Operation	Description	Assembler	Cycles
Absolute value	of float	VABS.F32	1
Addition	floating point	VADD.F32	1
Compare	float with register or zero	VCMP.F32	1
	float with register or zero	VCMP.E.F32	1
Convert	between integer, fixed-point, half-precision and float	VCVT.F32	1
Divide	Floating-point	VDIV.F32	14
Load	multiple doubles	VLDM.64	1+2*N, where N is the number of doubles.
	multiple floats	VLDM.32	1+N, where N is the number of floats.
	single double	VLDR.64	3
	single float	VLDR.32	2

Floating point instruction set cont.

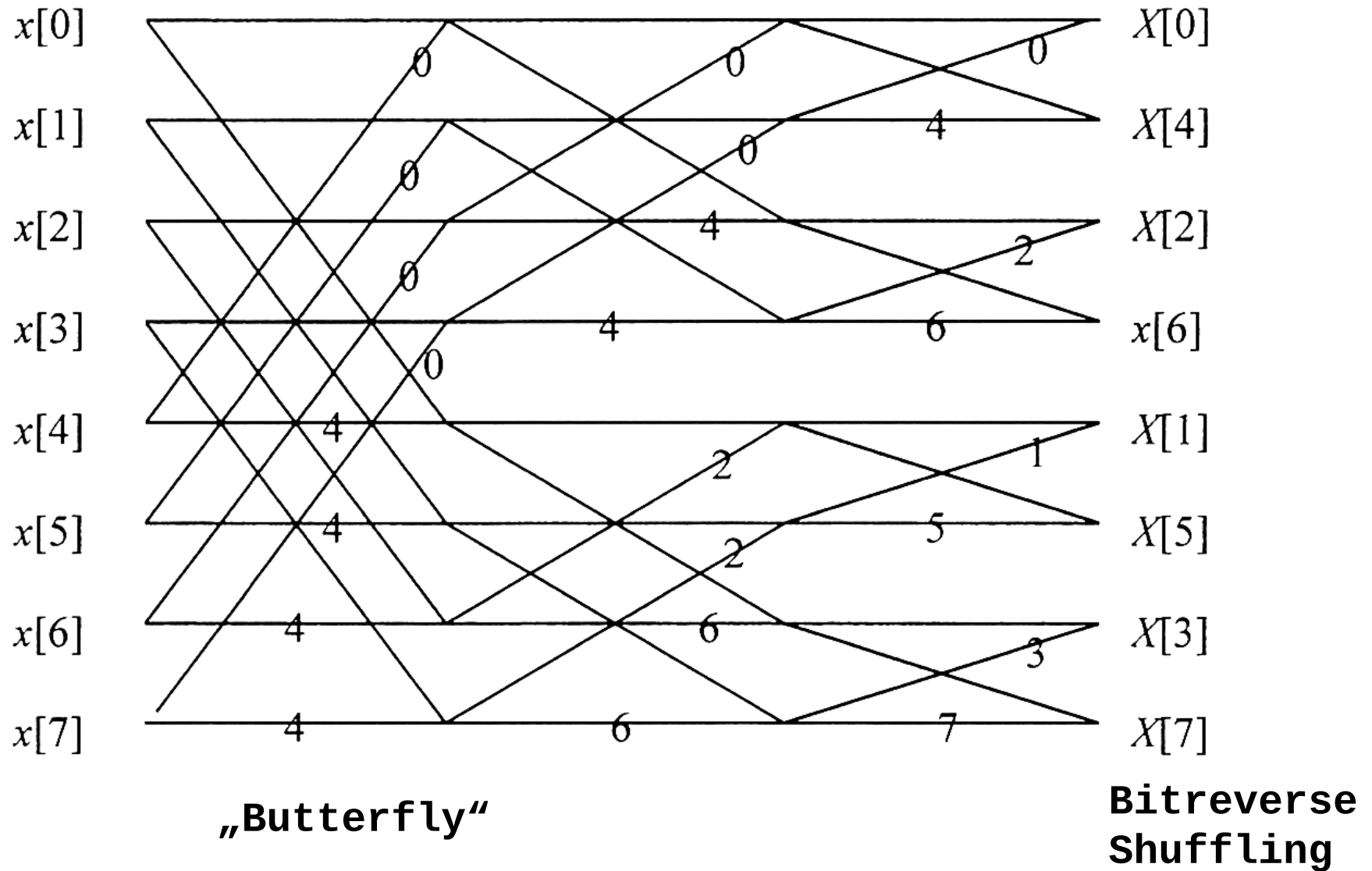
Operation	Description	Assembler	Cycles
Pop	double registers from stack	VPOP.64	$1+2*N$, where N is the number of double registers.
	float registers from stack	VPOP.32	$1+N$ where N is the number of registers.
Push	double registers to stack	VPUSH.64	$1+2*N$, where N is the number of double registers.
	float registers to stack	VPUSH.32	$1+N$, where N is the number of registers.
Square-root	of float	VSQRT.F32	14
Store	multiple double registers	VSTM.64	$1+2*N$, where N is the number of doubles.
	multiple float registers	VSTM.32	$1+N$, where N is the number of floats.
	single double register	VSTR.64	3
	single float registers	VSTR.32	2
Subtract	float	VSUB.F32	1

Operation	Description	Assembler	Cycles
Move	top/bottom half of double to/from core register	VMOV	1
	immediate/float to float-register	VMOV	1
	two floats/one double to/from two core registers or one float to/from one core register	VMOV	2
	floating-point control/status to core register	VMRS	1
	core register to floating-point control/status	VMSR	1
Multiply	float	VMUL.F32	1
	then accumulate float	VMLA.F32	3
	then subtract float	VMLS.F32	3
	then accumulate then negate float	VNMLA.F32	3
	then subtract then negate float	VNMLS.F32	3
Multiply (fused)	then accumulate float	VFMA.F32	3
	then subtract float	VFMS.F32	3
	then accumulate then negate float	VFNMA.F32	3
	then subtract then negate float	VFNMS.F32	3
Negate	float	VNEG.F32	1
	and multiply float	VNMUL.F32	1

CM4F floating-point arithmetics

- Multiplication is (usually) safe.
- Addition / subtraction may lead to information / precision loss.
- Configurable handling of overflow / underflow situations.
- The FPU does not have user mode exception traps. WRONG !
- Exception handling is tricky: Lazy stacking can be supported.

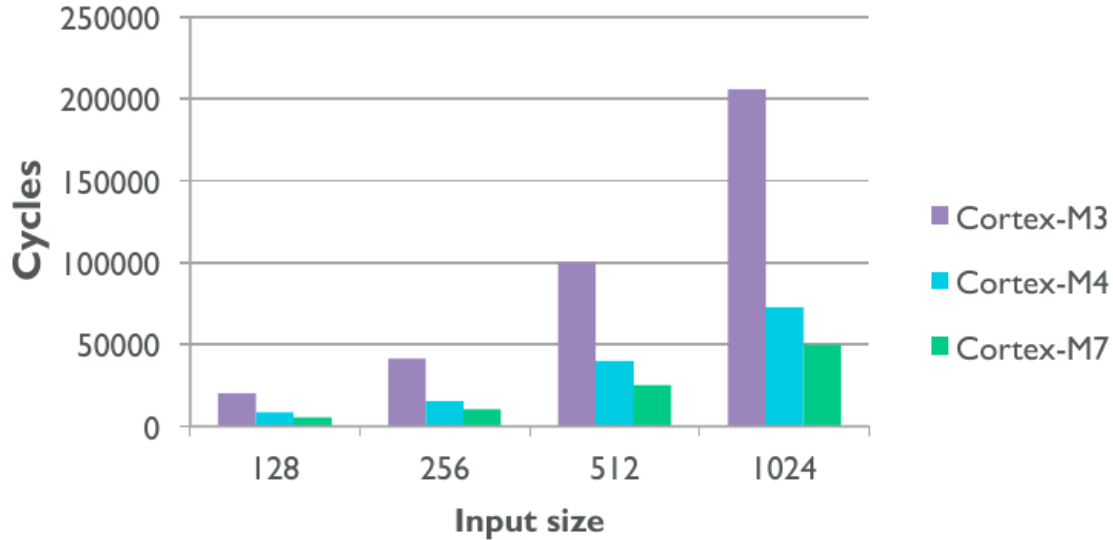
Cooley - Tukey - FFT



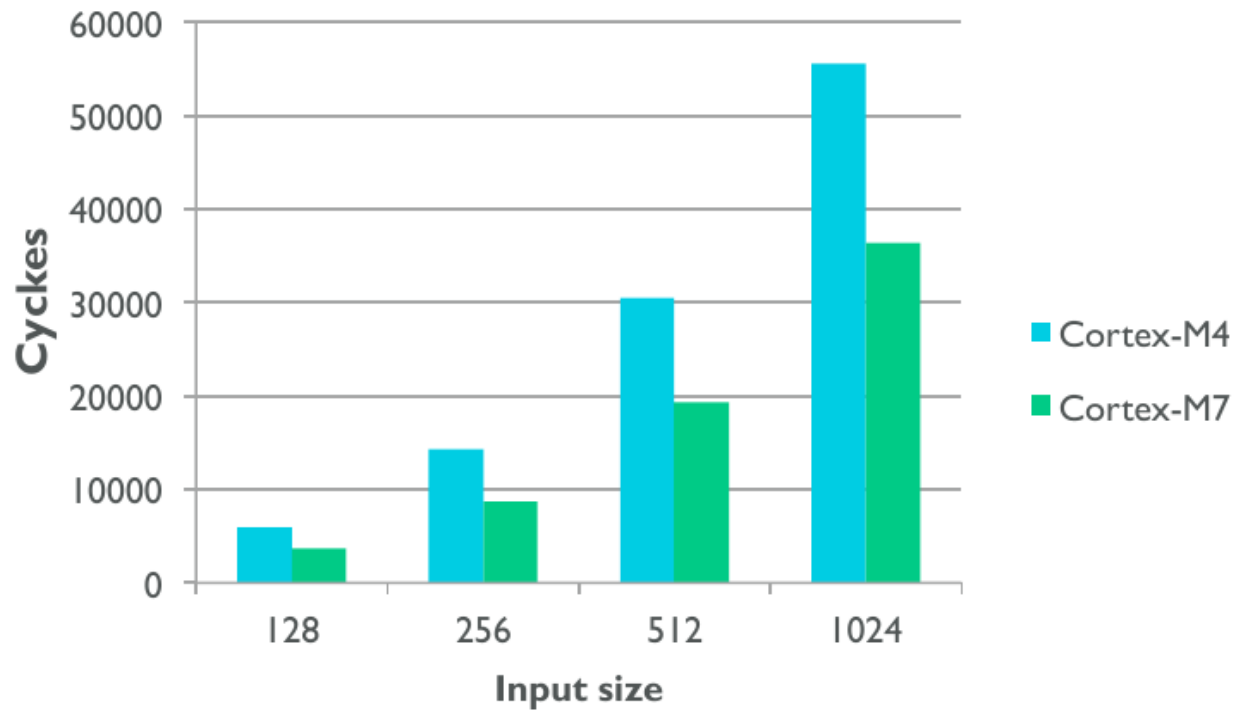
N^2 Products $\rightarrow \frac{1}{2} * N * \lg(N)$ Example $N=4096$ $17 * 10^6 \rightarrow 24 * 10^3$ complex ops

DSP Performance

RFFT Q31



RFFT F32



About the precision of “float”

```
volatile float a,b,c,d;
```

```
void ftest(void)
```

```
{
```

```
    a = 1.2345678901234567890;           // 1.23456788
```

```
    b = 1.2345678901234567890e-4;
```

```
    c = a + b;
```

```
    d = c - a;                           // 0.000123500824
```

```
    a = 1.2345678901234567890;
```

```
    b = 1.2345678901234567890e-4;
```

```
    c = a * b;
```

```
    d = c / a;                           // d = 0.00012345679
```

```
}
```

Just 4 significant digits
remaining !

All ~8 significant digits
remaining !

→ Denormalization during “+” and “-” !

Numerical Integration

```
void cp(void)
{
    for( float step=0.5f; step > 7e-8f; step *= 0.5f)
    {
#ifdef PRINT
        semihost.puts("step = 1 / ");
        semihost.puti( (uint32_t)( 1.0f / step));
        semihost.puts("      diff = ");
        volatile uint64_t start_time = getTime_usec_privileged();
#endif

        float y1;
        float sum = 0.0f;
        float y = 1.0f;
        for(float x=step; !( x > 1.0f); x+=step)
        {
            y1 = y;
            y = VSQRTF( 1.0f - x*x);
            sum += 0.5f * (y+y1);
        }
        sum *= step;

#ifdef PRINT
        volatile uint64_t stop_time = getTime_usec_privileged();

        semihost.puti( (int32_t) (((sum * xPi) -1.0f) * 1e6f) );
        semihost.puts(" ppm      runtime = ");
        semihost.puti( (uint32_t)(stop_time - start_time));
        semihost.puts(" micro-seconds \r\n");
#endif
    }
}
```


step = 1 / 2 diff = -130361 ppm runtime = 2 micro-seconds
step = 1 / 4 diff = -46436 ppm runtime = 2 micro-seconds
step = 1 / 8 diff = -16480 ppm runtime = 3 micro-seconds
step = 1 / 16 diff = -5837 ppm runtime = 5 micro-seconds
step = 1 / 32 diff = -2065 ppm runtime = 7 micro-seconds
step = 1 / 64 diff = -730 ppm runtime = 12 micro-seconds
step = 1 / 128 diff = -258 ppm runtime = 23 micro-seconds
step = 1 / 256 diff = -91 ppm runtime = 43 micro-seconds
step = 1 / 512 diff = -31 ppm runtime = 85 micro-seconds
step = 1 / 1024 diff = -11 ppm runtime = 167 micro-seconds
step = 1 / 2048 diff = -4 ppm runtime = 332 micro-seconds
step = 1 / 4096 diff = -1 ppm runtime = 674 micro-seconds
step = 1 / 8192 diff = 0 ppm runtime = 1323 micro-seconds
step = 1 / 16384 diff = 0 ppm runtime = 2644 micro-seconds
step = 1 / 32768 diff = 1 ppm runtime = 5284 micro-seconds
step = 1 / 65536 diff = 1 ppm runtime = 10568 micro-seconds
step = 1 / 131072 diff = 6 ppm runtime = 21133 micro-seconds
step = 1 / 262144 diff = -23 ppm runtime = 42263 micro-seconds
step = 1 / 524288 diff = 99 ppm runtime = 84524 micro-seconds
step = 1 / 1048576 diff = -271 ppm runtime = 169046 micro-seconds
step = 1 / 2097152 diff = 105 ppm runtime = 338089 micro-seconds
step = 1 / 4194304 diff = -2350 ppm runtime = 676175 micro-seconds
step = 1 / 8388608 diff = 30750 ppm runtime = 1352349 micro-seconds

Comparing CM4 and CM7

step = 1 / 2	diff = -130361 ppm	runtime = 18 micro-seconds
step = 1 / 4	diff = -46436 ppm	runtime = 40 micro-seconds
step = 1 / 8	diff = -16480 ppm	runtime = 85 micro-seconds
step = 1 / 16	diff = -5837 ppm	runtime = 172 micro-seconds
step = 1 / 32	diff = -2065 ppm	runtime = 348 micro-seconds
step = 1 / 64	diff = -730 ppm	runtime = 699 micro-seconds
step = 1 / 128	diff = -258 ppm	runtime = 1399 micro-seconds
step = 1 / 256	diff = -91 ppm	runtime = 2796 micro-seconds
step = 1 / 512	diff = -32 ppm	runtime = 5591 micro-seconds
step = 1 / 1024	diff = -11 ppm	runtime = 11161 micro-seconds
step = 1 / 2048	diff = -4 ppm	runtime = 22319 micro-seconds
step = 1 / 4096	diff = -1 ppm	runtime = 44652 micro-seconds
step = 1 / 8192	diff = 0 ppm	runtime = 89282 micro-seconds
step = 1 / 16384	diff = 0 ppm	runtime = 178575 micro-seconds
step = 1 / 32768	diff = 0 ppm	runtime = 357095 micro-seconds
step = 1 / 65536	diff = 0 ppm	runtime = 714186 micro-seconds
step = 1 / 131072	diff = 0 ppm	runtime = 1428129 micro-seconds
step = 1 / 262144	diff = 0 ppm	runtime = 2855990 micro-seconds
step = 1 / 524288	diff = 0 ppm	runtime = 5711562 micro-seconds
step = 1 / 1048576	diff = 0 ppm	runtime = 11422791 micro-seconds
step = 1 / 2097152	diff = 0 ppm	runtime = 22844415 micro-seconds
step = 1 / 4194304	diff = 0 ppm	runtime = 45687856 micro-seconds
step = 1 / 8388608	diff = 0 ppm	runtime = 91372910 micro-seconds
step = 1 / 16777216	diff = 0 ppm	runtime = 182743030 micro-seconds
step = 1 / 33554432	diff = 0 ppm	runtime = 365485955 micro-seconds

step = 1 / 2	diff = -130361 ppm	runtime = 3 micro-seconds
step = 1 / 4	diff = -46436 ppm	runtime = 2 micro-seconds
step = 1 / 8	diff = -16480 ppm	runtime = 4 micro-seconds
step = 1 / 16	diff = -5837 ppm	runtime = 7 micro-seconds
step = 1 / 32	diff = -2065 ppm	runtime = 13 micro-seconds
step = 1 / 64	diff = -730 ppm	runtime = 24 micro-seconds
step = 1 / 128	diff = -258 ppm	runtime = 47 micro-seconds
step = 1 / 256	diff = -91 ppm	runtime = 94 micro-seconds
step = 1 / 512	diff = -32 ppm	runtime = 186 micro-seconds
step = 1 / 1024	diff = -11 ppm	runtime = 370 micro-seconds
step = 1 / 2048	diff = -4 ppm	runtime = 747 micro-seconds
step = 1 / 4096	diff = -1 ppm	runtime = 1478 micro-seconds
step = 1 / 8192	diff = 0 ppm	runtime = 2953 micro-seconds
step = 1 / 16384	diff = 0 ppm	runtime = 5904 micro-seconds
step = 1 / 32768	diff = 0 ppm	runtime = 11810 micro-seconds
step = 1 / 65536	diff = 0 ppm	runtime = 23619 micro-seconds
step = 1 / 131072	diff = 0 ppm	runtime = 47234 micro-seconds
step = 1 / 262144	diff = 0 ppm	runtime = 94469 micro-seconds
step = 1 / 524288	diff = 0 ppm	runtime = 188941 micro-seconds
step = 1 / 1048576	diff = 0 ppm	runtime = 377881 micro-seconds
step = 1 / 2097152	diff = 0 ppm	runtime = 755758 micro-seconds
step = 1 / 4194304	diff = 0 ppm	runtime = 1511517 micro-seconds
step = 1 / 8388608	diff = 0 ppm	runtime = 3023028 micro-seconds
step = 1 / 16777216	diff = 0 ppm	runtime = 6046052 micro-seconds
step = 1 / 33554432	diff = 0 ppm	runtime = 12092096 micro-seconds

—► CM7 is 30 times faster than the CM4 with it's double-precision emulation !

ARM Cortex M DSP Library

Main Page	Related Pages	Modules	Data Structures ▾	Files ▾	
---------------------------	-------------------------------	-------------------------	-----------------------------------	-------------------------	--

Modules

Here is a list of all modules:

- | | |
|---|--|
| ▶ Basic Math Functions | |
| ▶ Fast Math Functions | |
| ▶ Complex Math Functions | |
| ▶ Filtering Functions | |
| ▶ Matrix Functions | |
| ▶ Transform Functions | |
| ▶ Controller Functions | |
| ▶ Statistics Functions | |
| ▶ Support Functions | |
| ▶ Interpolation Functions | |
| Examples | |
| CMSIS Core Instruction Interface | |
| CMSIS SIMD Intrinsics | |
| CMSIS Global Defines | |
| ▶ Defines and Type Definitions | |
| ▶ Functions and Instructions Reference | |



https://arm-software.github.io/CMSIS_5/DSP/html/index.html

Example: C++ Wrapper Class

```
#ifndef FLOAT3MATRIX_H_
#define FLOAT3MATRIX_H_

#include "arm_math.h"
#include "matrix.h"
#include "float3vector.h"

//! matrix of 3 float values
class float3matrix: public matrix<float, 3>
{
    //friend class float3vector;
public:
    float3matrix( float *init=0):
    matrix<float, 3>( init)
    {
        ami.numCols=3;
        ami.numRows=3;
        ami.pData=(float*)matrix<float, 3>::e;
    }
    void transpose( float3matrix &result);
    float3vector operator *( const float3vector& right) const;
private:
    arm_matrix_instance_f32 ami;

```

Object from CMSIS DSP Lib

Using the DSP Library

```
#include "my_assert.h"
#include "float3matrix.h"
#include "arm_math.h"

void float3matrix::transpose( float3matrix &result)
{
    arm_matrix_instance_f32 from, to;
    from.numCols=3;
    from.numRows=3;
    from.pData=(float *) (this->e);
    to.numCols=3;
    to.numRows=3;
    to.pData=(float *) (&(result.e));
    (void)arm_mat_trans_f32( &from, &to);
}

float3vector float3matrix::operator *( const float3vector& right) const
{
    float3vector ret;
    arm_matrix_instance_f32 ret_ami;
    ret_ami.numCols=1;
    ret_ami.numRows=3;
    ret_ami.pData=ret.e;
    arm_matrix_instance_f32 input;
    input.numCols=1;
    input.numRows=3;
    input.pData=(float *) (right.e);
    (void)arm_mat_mult_f32( &ami, &input, &ret_ami);
    return ret;
}
```

DSP-Lib-based PID-Controller

```
class PID_controller
{
public:

    inline PID_controller( float p=0.0, float i=0.0 , float d=0.0, float sampling_time=1.0)
    {
        pid.Kp = p;
        pid.Ki = sampling_time / i;
        pid.Kd = d / sampling_time;
        arm_pid_init_f32( &pid, true);
    };
    inline float step( float setpoint, float actual_value)
    {
        return arm_pid_f32( &pid, setpoint - actual_value);
    };
    inline float out( void ) const
    {
        return pid.state[2];
    }
    inline void reset( void)
    {
        arm_pid_reset_f32( &pid);
    };
    inline void set_integrator( float value = 0.0f)
    {
        pid.state[2]=value;
    };

private:
    arm_pid_instance_f32 pid;
};
```

Lecture Overview

- Introduction, Microcontroller basics
- 32-bit Microcontroller technology
- ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
- Cortex M Microcontroller Instruction Set
- Exception Handling, Interrupts
- Real-time Operating Systems (RTOS)
- Memory Management, protected memory, MPU
- Implementing Calculations, Data Formats, FPU
- I/O, Device drivers
- Application Design
- Case Studies, Advanced Design Patterns

<https://skript.eit.h-da.de/ams/>

